

StockAgent.NET 项目实施说明

生成日期：2026-06-23

1. 项目目标

StockAgent.NET 是一个面向 A 股和港股的股票深度研究 Agent MVP。用户在 Web UI 中输入股票代码并选择市场后，系统会创建研究任务，后台异步执行研究流水线，最终生成中文评分/评级型研究报告，并预留 PDF 导出能力。

当前版本已经形成“真实数据源网关 + 用户配置 + 固定流程多 Agent 分析”的纵向切片。行情/公告可以通过自定义 HTTP 数据源网关接入，模型分析通过用户配置的 OpenAI-compatible API Key 调用，向量检索仍作为后续增强项保留。

2. 当前技术栈

后端：

- .NET 10
- ASP.NET Core Web API / Minimal API
- Entity Framework Core 10
- Npgsql.EntityFrameworkCore.PostgreSQL
- PostgreSQL
- System.Threading.Channels
- Microsoft.SemanticKernel
- Microsoft.Playwright
- xUnit
- FluentAssertions
- Microsoft.AspNetCore.Mvc.Testing
- Microsoft.EntityFrameworkCore.InMemory

前端：

- React 19
- TypeScript 6
- Vite 8
- TanStack Query
- React Router
- Microsoft SignalR JavaScript client
- Vitest / Testing Library / jsdom

本地运行：

- docker-compose.yml 提供 PostgreSQL 配置
- API 开发端口：`http://localhost:5000`
- Web 开发端口：`http://127.0.0.1:5173`

3. 项目结构

核心目录如下：

StockAgent.NET/	
src/	
StockAgent.Api/	后端 API、领域模型、编排器、持久化、PDF、报告生成
StockAgent.Web/	React + Vite 前端工作台
tests/	
StockAgent.Api.Tests/	后端单元测试和 API 集成测试
docs/	
superpowers/specs/	设计方案
superpowers/plans/	实施计划
docker-compose.yml	本地 PostgreSQL 配置

后端按职责拆分为：

- `Domain`：研究任务、证据、文档、报告、导出记录、枚举状态。
- `Features`：Minimal API endpoint，按业务能力分组。
- `Infrastructure/Persistence`：EF Core DbContext。
- `Infrastructure/Queueing`：后台任务队列和 worker。
- `Infrastructure/Research`：研究流水线编排器。
- `Infrastructure/DataSources`：行情和公共资料 provider 抽象，支持 fake fallback 和用户配置的自定义 HTTP 数据源。
- `Infrastructure/Documents`：文档切片和上下文预算控制。
- `Infrastructure/Ai`：Semantic Kernel 聊天边界、固定流程多 Agent 分析服务、Agent 上下文预算和输出校验。
- `Infrastructure/Reports`：报告生成。
- `Infrastructure/Pdf`：PDF 导出边界。

前端按职责拆分为：

- `api/researchApi.ts`：调用后端 API。
- `models.ts`：前端共享类型。
- `components/ResearchWorkbench.tsx`：主工作台。
- `components/TaskTimeline.tsx`：研究进度时间线。
- `components/ReportViewer.tsx`：报告展示和 PDF 导出入口。
- `components/EvidenceDrawer.tsx`：证据列表。
- `components/SettingsPage.tsx`：首版配置展示。

4. 总体架构

整体架构是一个模块化单体：

```
React Web UI
|
| HTTP / JSON
v
ASP.NET Core Minimal API
|
| EF Core
v
PostgreSQL
```

后台任务：

```
Create Task API
-> ResearchTaskQueue
-> ResearchWorker
-> ResearchOrchestrator
-> ConfiguredMarketDataProvider / ConfiguredWebResearchProvider
-> DocumentChunker / ContextBudgetManager
-> SemanticKernelResearchAnalysisService
-> ReportGenerator
-> ResearchReport / EvidenceCard 持久化
```

前端不直接执行研究逻辑，只负责提交任务、轮询任务状态、读取报告和证据。后端通过队列和 hosted service 异步处理研究任务，避免 HTTP 请求长时间阻塞。

5. 后端启动与依赖注册

入口文件是 `src/StockAgent.Api/Program.cs`。它完成以下工作：

1. 注册 OpenAPI。
2. 注册 CORS，允许本地 Vite 前端访问 API。
3. 配置 JSON enum 以字符串形式输出。
4. 注册 `StockAgentDbContext`，使用 PostgreSQL。
5. 注册研究任务队列 `IResearchTaskQueue`。
6. 注册 fake fallback 和用户配置型行情/证据 provider。
7. 注册文档切片器、上下文预算管理器、研究编排器。
8. 注册 Semantic Kernel 聊天客户端、Agent 上下文预算器和 `IResearchAnalysisService`。
9. 注册报告生成器和 PDF 导出服务。
10. 注册后台 worker `ResearchWorker`。
11. 在 Development 环境下对关系型数据库执行 EF Core migrations。
12. 映射研究任务、报告、证据、PDF、设置和健康检查端点。

当前项目已经使用 EF Core migrations 记录数据库结构演进。正式部署时应在启动或发布流程中执行 migrations，确保 PostgreSQL schema 与代码模型一致。

6. 数据模型

主要领域实体如下：

ResearchTask

研究任务根实体，记录：

- 标准化股票代码，如 `600519.SH`、`00700.HK`
- 市场：A 股或港股
- 公司名称
- 任务状态
- 当前阶段
- 进度百分比
- 报告语言
- 创建/更新时间
- 阶段步骤列表

ResearchStep

研究流水线每个阶段的审计记录，记录阶段名、状态、开始/完成时间、输入输出摘要和错误信息。

DocumentSource

收集到的原始资料来源，记录 URL、标题、来源类型、发布日期、内容 hash。DbContext 中对 `(ResearchTaskId, ContentHash)` 建唯一索引，用来避免同一任务内重复资料。

DocumentChunk

文档切片结果，记录所属文档、切片序号、文本和粗略 token 估算。DbContext 中对 `(DocumentSourceId, ChunkIndex)` 建唯一索引。

EvidenceCard

压缩后的证据卡，是报告和分析真正使用的上下文单元。它记录 claim、snippet、confidence、relevance、来源时间和报告章节。

ResearchReport

最终研究报告，记录 Markdown、HTML、评分 JSON、数据截止时间、模型 provider 和模型名。

PdfExport / ModelInvocation / AppSetting

这些实体为后续 PDF 导出审计、模型调用审计和应用设置持久化预留。

7. 股票代码归一化

实现文件: `src/StockAgent.Api/Features/ResearchTasks/TickerNormalizer.cs`

功能规则:

- `600519` 会归一化为 `600519.SH`
- 非 6 开头的 6 位 A 股代码默认使用 `.SZ`
- `700` 搭配港股市场提示会归一化为 `00700.HK`
- 已带 `.SH`、`.SZ`、`.HK` 后缀的代码会转为大写并保留
- 不支持或模糊输入会抛出 `ArgumentException`

对应测试文件: `tests/StockAgent.Api.Tests/TickerNormalizerTests.cs`

8. API 端点

研究任务端点

文件: `src/StockAgent.Api/Features/ResearchTasks/ResearchTaskEndpoints.cs`

- `POST /api/research-tasks`: 创建研究任务, 归一化 ticker, 保存数据库, 放入后台队列。
- `GET /api/research-tasks`: 按创建时间倒序列出任务。
- `GET /api/research-tasks/{id}`: 读取单个任务。

报告端点

文件: `src/StockAgent.Api/Features/Reports/ReportEndpoints.cs`

- `GET /api/research-tasks/{id}/report`: 读取任务生成的研究报告。

证据端点

文件: `src/StockAgent.Api/Features/Evidence/EvidenceEndpoints.cs`

- `GET /api/research-tasks/{id}/evidence`: 读取任务对应的证据卡。

PDF 端点

文件: `src/StockAgent.Api/Features/Pdf/PdfEndpoints.cs`

- `POST /api/research-tasks/{id}/pdf`: 读取报告 HTML 并调用 PDF 服务导出。

设置和健康检查端点

文件:

- `src/StockAgent.Api/Features/Settings/SettingsEndpoints.cs`
- `src/StockAgent.Api/Features/Health/DataSourceHealthEndpoints.cs`

当前返回固定的首版配置和 fake provider 健康状态。

9. 后台队列和 Worker

队列接口: `IResearchTaskQueue`

队列实现: `ResearchTaskQueue`

worker: `ResearchWorker`

当前队列使用 `System.Threading.Channels`，是一个进程内无界队列。创建研究任务后，API 保存 `ResearchTask`，然后将任务 ID 写入队列。`ResearchWorker` 作为 ASP.NET Core hosted service 持续消费队列，每取到一个任务 ID，就创建 DI scope 并调用 `ResearchOrchestrator.RunAsync()`。

这种方式适合 MVP，因为实现简单且便于测试。生产环境如果要支持多实例部署、失败恢复和任务重试，建议换成持久化队列或后台任务系统，例如 Hangfire、Quartz.NET、MassTransit 或 PostgreSQL-backed queue。

10. 研究流水线编排

核心文件: `src/StockAgent.Api/Infrastructure/Research/ResearchOrchestrator.cs`

当前流水线如下：

```

读取 ResearchTask
-> 状态改为 CollectStructuredData
-> 调用 IMarketDataProvider 获取结构化行情快照
-> 状态改为 CollectPublicEvidence
-> 调用 IWebResearchProvider 获取公共资料
-> 状态改为 IngestAndIndexDocuments
-> 将资料保存为 DocumentSource
-> 使用 DocumentChunker 切片
-> 将切片保存为 DocumentChunk
-> 生成 EvidenceCard
-> 状态改为 AnalyzeWithSemanticKernel
-> 使用 ContextBudgetManager 选择 Top evidence
-> 调用 IResearchAnalysisService 分析
-> 状态改为 GenerateReport
-> 调用 ReportGenerator 生成 Markdown/HTML/评分
-> 保存 ResearchReport
-> 状态改为 Ready, 进度 100
  
```

当前 provider 采用可配置策略：

- 未配置自定义数据源时，使用 `FakeMarketDataProvider` 和 `FakeWebResearchProvider` 保证本地测试稳定。
- 配置 `CustomHttp` 后，通过外部 FastAPI 数据源网关获取真实行情、财务快照、公告和公开证据。

这样既可以在没有外部依赖时跑通测试，也可以在生产配置中接入真实 A 股/港股数据。

11. 上下文控制和向量能力现状

当前已经实现了两个上下文控制基础件：

- `DocumentChunker`：把长文本切成固定字符上限的块，并估算 token。
- `ContextBudgetManager`：按 relevance 和 confidence 排序，选出 Top-K 证据卡。

当前还没有接入向量检索。MVP 可以先不使用向量，因为 fake provider 返回资料很少，直接证据卡排序即可。

真正 deep research 阶段建议加入：

```
DocumentChunk
-> 生成 embedding
-> 存入 PostgreSQL + pgvector
-> 按报告章节或问题召回 Top-K
-> rerank
-> 形成 evidence pack
-> 交给 Semantic Kernel / LLM
```

推荐优先使用 PostgreSQL + pgvector，因为项目已经使用 PostgreSQL，不需要一开始引入独立向量数据库。

12. 固定流程多 Agent 与 Semantic Kernel 边界

接口：`IResearchAnalysisService`

实现：`SemanticKernelResearchAnalysisService`

当前分析服务已经从确定性评分升级为固定流程多 Agent。`.NET ResearchOrchestrator` 仍然负责可靠编排和状态持久化，LLM Agent 只负责各自阶段的分析判断。

当前定义的角色包括：

- `MarketFinancialAgent`：分析结构化行情、估值、市值、收入增长和净利率。
- `EvidenceFilingAgent`：分析受限
`EvidencePack`，提取公告和公开证据中的正面事实、负面事实、不确定性和引用。
- `SynthesisReportAgent`：综合前两个 Agent
的结构化输出，生成评分、风险等级、估值判断、关键假设、关键结论和中文 Markdown 报告。
- `ReviewAgent`：检查报告是否存在无证据结论、直接买卖建议、过度确定性表达或引用缺失。

固定流程如下：

```
MarketFinancialAgent
  \
  -> SynthesisReportAgent -> ReviewAgent -> ReportGenerator
  /
EvidenceFilingAgent
```

模型调用通过 `IModelChatClient` 抽象，真实实现是
`SemanticKernelModelChatClient`。它读取当前用户保存的模型配置，包括 provider、base URL、model
和解密后的 API Key，然后通过 Semantic Kernel 调用 OpenAI-compatible chat completion。

每次 Agent 调用都会写入 `ModelInvocations`，记录：

- 研究任务 ID
- Agent 名称
- Provider
- Model
- 耗时
- 状态

Agent 上下文管理

LLM Agent 不直接读取无限长度的原始文档。上下文管理由 `AgentContextBudgeter` 在进入模型前完成：

- `MarketFinancialAgent` 只接收结构化行情财务摘要，不接收完整财报表格。
- `EvidenceFilingAgent` 只接收按相关性和置信度筛选后的 `EvidencePack`，不直接阅读公告全文。
- `SynthesisReportAgent` 只接收前两个 Agent 的结构化摘要和受限引用。
- `ReviewAgent` 只接收报告草稿、关键结论 `keyClaims` 和受限 `citations`。

`AgentOutputValidators` 会在阶段之间做确定性校验，例如分数是否在 0-100、证据引用是否指向真实 evidence card、关键结论是否绑定证据 ID。这样可以避免后一个 Agent 消费无效或过大的上下文。

13. 报告生成

文件：`src/StockAgent.Api/Infrastructure/Reports/ReportGenerator.cs`

报告生成器把结构化分析结果转换为：

- Markdown 正文
- HTML 正文
- `ReportScore` 评分对象

如果 `AiAnalysisResult.ReportMarkdown` 存在，报告生成器会优先使用 `SynthesisReportAgent` 生成的 Markdown；如果没有，则回退到后端固定模板。

报告结构包括：

- 标题
- 评分结论
- 核心摘要
- 关键假设
- 来源证据
- 风险提示

当前报告明确写入“本报告仅用于研究辅助，不构成买卖建议”，避免直接形成买卖指令。

14. PDF 导出

接口: `IPdfExportService`

实现: `PlaywrightPdfExportService`

PDF 导出逻辑:

1. 读取报告 HTML。
2. 组合一个稳定的中文打印 HTML wrapper。
3. 使用 Playwright Chromium 打开页面内容。
4. 调用 `PdfAsync` 输出 A4 PDF。
5. 返回 PDF 文件路径。

注意: 当前机器上 Playwright Chromium 下载曾超时, 因此代码已存在, 但实际导出前需要成功执行:

```
powershell -ExecutionPolicy Bypass -File src\StockAgent.Api\bin\Debug\net10.0\playwright.ps1 install chromium
```

15. 前端实现

入口:

- `src/StockAgent.Web/src/main.tsx`
- `src/StockAgent.Web/src/App.tsx`

主组件: `ResearchWorkbench`

前端工作流:

1. 默认输入 `00700.HK`, 市场为港股。
2. 用户点击 “开始研究”。
3. `createResearchTask()` 调用 `POST /api/research-tasks`。
4. 成功后记录 selected task id。
5. `listResearchTasks()` 每 3 秒轮询任务列表。
6. 当任务状态为 `Ready` 或 `Completed` 时, 读取报告。
7. 同时读取证据卡。
8. 用户可以点击 “导出 PDF” 触发 PDF API。

TanStack Query 负责:

- 任务列表轮询
- 创建任务 mutation
- 报告读取 query

- 证据读取 query
- PDF 导出 mutation

16. 本地运行方式

如果本机有 Docker:

```
docker compose up -d
dotnet run --project src/StockAgent.Api --launch-profile http
```

另一个终端启动前端:

```
cd src/StockAgent.Web
npm run dev -- --host 127.0.0.1 --port 5173
```

访问:

```
http://127.0.0.1:5173/
```

如果没有 Docker, 需要本地安装 PostgreSQL, 并创建:

```
CREATE USER stockagent WITH PASSWORD 'stockagent';
CREATE DATABASE stockagent OWNER stockagent;
```

连接串位于:

```
src/StockAgent.Api/appsettings.Development.json
```

当前值:

```
Host=localhost;Port=5432;Database=stockagent;Username=stockagent;Password=stockagent
```

17. 测试覆盖

当前测试位于 `tests/StockAgent.Api.Tests`:

- `TickerNormalizerTests`: 验证 A 股/港股代码归一化。
- `ResearchTaskQueueTests`: 验证队列入队和出队。
- `ResearchTaskApiTests`: 验证创建研究任务 API、字符串枚举响应和 EF InMemory 替换。
- `StockAgentDbContextTests`: 验证 EF 模型和关键索引。
- `DocumentChunkerTests`: 验证长文本切片不超过上限。
- `ContextBudgetManagerTests`: 验证按相关性选取证据卡。

常用验证命令:

```
dotnet build StockAgent.sln
dotnet test StockAgent.sln
cd src/StockAgent.Web
npm run build
```

18. 当前限制

当前版本仍有这些限制：

- 自定义 HTTP 数据源和模型 API Key 需要用户在设置页配置；未配置时仍会使用 fake fallback 或在模型分析阶段失败。
- 没有接入向量检索和 pgvector。
- 进程内队列不适合多实例生产部署。
- PDF 依赖 Playwright Chromium，本机需要先安装浏览器二进制。
- 前端当前以轮询为主，SignalR 包已安装但还没有接实时推送。
- 报告 HTML 当前是简单编码换行，不是完整 Markdown 渲染器。

19. 推荐下一步

建议按下面顺序增强：

1. 接入真实 PostgreSQL 并跑通端到端流程。
2. 添加 EF Core migrations。
3. 部署并稳定运行真实 A 股/港股 FastAPI 数据源网关。
4. 增强公告正文抽取、年报下载和证据质量。
5. 加入 pgvector，建立 DocumentEmbedding 表。
6. 实现 EvidenceRetrievalService，支持章节级召回。
7. 为多 Agent 增加失败重试和报告修订机制。
8. 加 SignalR 进度推送，减少前端轮询。
9. 增强 PDF 导出和报告版式。
10. 增加更多端到端 API 测试，覆盖真实数据源和真实模型配置。

20. 关键文件索引

后端入口：

- ``src/StockAgent.Api/Program.cs``

持久化：

- ``src/StockAgent.Api/Infrastructure/Persistence/StockAgentDbContext.cs``

研究任务：

- ``src/StockAgent.Api/Features/ResearchTasks/ResearchTaskEndpoints.cs``
- ``src/StockAgent.Api/Features/ResearchTasks/TickerNormalizer.cs``

后台任务：

- `src/StockAgent.Api/Infrastructure/Queueing/ResearchTaskQueue.cs`
- `src/StockAgent.Api/Infrastructure/Queueing/ResearchWorker.cs`

研究编排：

- `src/StockAgent.Api/Infrastructure/Research/ResearchOrchestrator.cs`

数据源：

- `src/StockAgent.Api/Infrastructure/DataSources/FakeMarketDataProvider.cs`
- `src/StockAgent.Api/Infrastructure/DataSources/FakeWebResearchProvider.cs`

上下文控制：

- `src/StockAgent.Api/Infrastructure/Documents/DocumentChunker.cs`
- `src/StockAgent.Api/Infrastructure/Documents/ContextBudgetManager.cs`

AI 边界：

- `src/StockAgent.Api/Infrastructure/Ai/IResearchAnalysisService.cs`
- `src/StockAgent.Api/Infrastructure/Ai/SemanticKernelResearchAnalysisService.cs`

报告和 PDF：

- `src/StockAgent.Api/Infrastructure/Reports/ReportGenerator.cs`
- `src/StockAgent.Api/Infrastructure/Pdf/PlaywrightPdfExportService.cs`

前端：

- `src/StockAgent.Web/src/components/ResearchWorkbench.tsx`
- `src/StockAgent.Web/src/api/researchApi.ts`
- `src/StockAgent.Web/src/models.ts`
- `src/StockAgent.Web/src/styles.css`

测试：

- `tests/StockAgent.Api.Tests`

21. 总结

这个项目当前已经不是单纯的设计稿，而是一个可以继续演进的研究 Agent MVP。它的核心价值在于边界已经拆开：

- UI 只负责交互。
- API 只负责创建和读取任务。

- 队列负责异步化。
- Orchestrator 负责编排阶段。
- Provider 负责数据来源。
- Document/Evidence 层负责上下文压缩。
- AI service 负责模型分析边界。
- Report/PDF 层负责输出。

后续真正要加强的是数据质量、检索能力、模型调用和生产级可靠性。架构上已经为这些增强留出了位置。